

Implementasi Enkripsi dan Dekripsi Teks

Menggunakan Algoritma AES-128 CBC

Christian Aprilio Sihite – 2401020151

Andrian Yuza Swanda – 2401020157

Muhammad Fauzi – 2401020160

Adhie Mulia Sembiring – 2401020165

Outline

01

Pendahuluan

Latar belakang, rumusan masalah, tujuan & manfaat

02

Tinjauan Pustaka

AES, CBCMode, PKCS#7Padding, penelitian terkait

03

Metodologi & Perancangan

Tahapan pengembangan dan desain sistem

04

Implementasi & Pengujian

Kode program, tampilan sistem, dan hasil uji

05

Penutup

Kesimpulan dan saran pengembangan

Pendahuluan

Latar Belakang

- Pertukaran data digital rentan terhadap penyadapan, pencurian, dan manipulasi data
- Kriptografi sebagai solusi menjaga kerahasiaan data (Confidentiality, Integrity, Availability)
- AES-128 adalah standar NIST (FIPS PUB 197) yang diakui secara internasional
- Implementasi mandiri memungkinkan pemahaman mendalam proses internal AES

Rumusan Masalah

- Bagaimana mengimplementasikan AES-128CBC secara mandiri dengan Python?
- Bagaimana membangun sistem enkripsi/dekripsi berbasis web dengan Gradio?
- Bagaimana menerapkan mekanisme CBC dan PKCS#7 Padding?
- Bagaimana sistem menangani input tidak valid dan error dekripsi?

Tujuan & Manfaat Proyek

Tujuan Proyek

1. Implementasi AES-128 CBC secara mandiri menggunakan Python
2. Membangun aplikasi enkripsi/dekripsi berbasis web dengan Gradio
3. Implementasi CBC Mode, IV acak, dan PKCS#7 Padding
4. Melakukan pengujian fungsional dan keamanan sistem
5. Memahami proses internal AES secara langsung

Manfaat Proyek

- Menambah pemahaman konsep keamanan informasi & kriptografi
- Media pembelajaran implementasi AES secara mandiri
- Pengalaman pengembangan aplikasi keamanan berbasis web
- Sarana pembelajaran proses enkripsi & dekripsi data
- Contoh implementasi keamanan informasi menggunakan Python

Advanced Encryption Standard (AES)

Jenis Kriptografi Simetris (Block Cipher)	Standard NIST FIPS PUB 197 (2001)	Ukuran Blok 128 bit (16 byte)	Kunci AES-128 → 16 byte 10 round
---	---	-------------------------------------	--

4 Transformasi Utama AES per Round:



CBC Mode: Setiap blok plaintext di-XOR dengan ciphertext sebelumnya sebelum dienkripsi. Blok pertama menggunakan **Initialization Vector (IV)** acak 16 byte sehingga ciphertext selalu berbeda meski plaintext & key sama.

Metodologi Pengembangan

01

Studi Literatur

Mempelajari referensi AES, CBC, PKCS#7 dari NIST FIPS PUB 197 dan jurnal ilmiah

02

Analisis Kebutuhan

Identifikasi kebutuhan fungsional & non-fungsional sistem enkripsi/dekripsi

03

Perancangan Sistem

Menyusun alur kerja, struktur modul AES, mekanisme CBC & desain UI Gradio

04

Implementasi

Translasi rancangan ke kode Python secara mandiri tanpa library AES siap pakai

05

Pengujian

Uji fungsional enkripsi, dekripsi, validasi input, error handling, dan multi-block

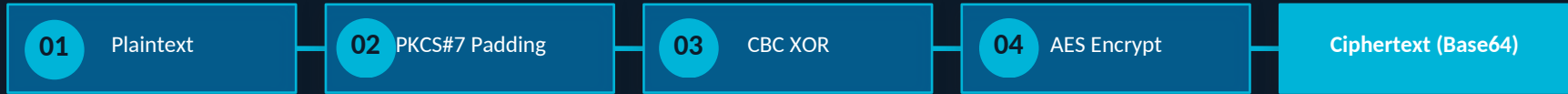
06

Evaluasi

Analisis hasil pengujian terhadap fungsionalitas, keamanan, dan kemudahan penggunaan

Perancangan Sistem

Alur Enkripsi:



Alur Dekripsi:



Aspek Keamanan yang Diterapkan:

AES-128 CBC	IV Acak 16B	PKCS#7 Padding	Validasi Input	Error Handling
Enkripsi standar internasional NIST	Ciphertext selalu unik tiap enkripsi	Penanganan plaintext sembarang panjang	Cek panjang key, format, input kosong	Deteksi key salah, format Base64 invalid

Implementasi Kriptografi

AES Core

- SubBytes (S-Box substitution)
- ShiftRows (baris siklis)
- MixColumns ($GF\ 2^8$)
- AddRoundKey (XOR state + key)

Key Expansion

- Menghasilkan 11 round key
- Dari kunci AES-128 (128-bit)
- Menggunakan RCON & S-Box
- Sesuai standar NIST FIPS 197

CBC + Padding

- IV acak 16 byte (os.urandom)
- XOR tiap blok dengan prev CT
- PKCS#7: pad agar 16-byte multiple
- Remove padding saat dekripsi

Antarmuka Gradio

- Input: plaintext & key (str)
- Output: ciphertext Base64
- Log proses AES per round
- Error handling terpadu

Fitur & Tampilan Sistem

Halaman Utama

Input plaintext & key AES-128 (16 karakter) dengan validasi otomatis

Proses Enkripsi

Output ciphertext dalam format Base64 dengan IV acak terembed

Proses Dekripsi

Pemulihan plaintext dari ciphertext Base64 menggunakan key yang sama

Log AES

Menampilkan proses per-round: state, round key, SubBytes, ShiftRows, dll.

Error Handling

Deteksi key salah, ciphertext rusak, padding invalid, Base64 tidak valid

Hasil Pengujian Sistem

SkenarioPengujian Fungsional:

No	Skenario Pengujian	Hasil
1	Enkripsi plaintext	✓ Berhasil
2	Dekripsi ciphertext	✓ Berhasil
3	Key salah	✗ Ditolak
4	Input kosong	✗ Ditolak
5	Ciphertext rusak	✗ Ditolak
6	Pengujian multi-block	✓ Berhasil
7	Pengujian IV acak	✓ Berhasil



7 dari 7 skenario berjalan sesuai harapan (100% sesuai ekspektasi), terdiri dari 4 skenario berhasil diproses dan 3 skenario berhasil ditolak sebagai bagian dari validasi keamanan.

Analisis Hasil

✓ Kelebihan Sistem

- Implementasi AES dilakukan secara mandiri (tanpa library)
- Antarmuka mudah digunakan melalui Gradio
- Mendukung CBC mode dengan IV acak tiap enkripsi
- Validasi input berjalan dengan baik
- Menampilkan log proses AES per round

⚠ Kekurangan & Limitasi

- Hanya mendukung AES-128 (belum 192 & 256)
- Hanya mendukung data berupa teks
- Belum mendukung upload dan enkripsi file
- Belum ada penyimpanan hasil enkripsi

Penutup

Kesimpulan

Algoritma AES-128 CBC berhasil diimplementasikan secara mandiri menggunakan Python.

Sistem berhasil melakukan enkripsi & dekripsi teks berbasis web menggunakan Gradio.

Implementasi CBC mode dan PKCS#7 Padding berhasil diterapkan sesuai standar AES.

Seluruh fitur sistem berjalan baik dan mampu menangani berbagai kondisi input.

Saran Pengembangan

- Menambahkan dukungan AES-192 & AES-256
- Mengembangkan antarmuka yang lebih interaktif
- Menambahkan fitur enkripsi file
- Melakukan pengujian performa pada data besar
- Menambahkan sistem autentikasi pengguna

Terima Kasih

Any Questions?